



Education
@topdev_media



React.js

React Router

CHEATSHEET

Save it for later !

INSTALLATION

To use React Router in your project, you need to install it via npm or yarn:

○ ○ ○

```
npm install react-router-dom
```

Make sure you **wrap your app in a BrowserRouter** for React Router to function properly.

CORE COMPONENTS

BrowserRouter

- Acts as the base for your routing.
- It listens to URL changes and renders the corresponding components.
- Place it at the top level of your application.

○ ○ ○

```
import { BrowserRouter } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
      </Routes>
    </BrowserRouter>
  );
}
```

DEFINING ROUTES

Basic Route

○ ○ ○

```
<Routes>
  <Route path="/" element={<Home />} />
</Routes>
```

- The **path** defines the URL for this route.
- The **element** specifies the component to render.

Wildcard Route

○ ○ ○

```
<Route path="*" element={<NotFound />} />
```

- The ***** acts as a wildcard to catch all unmatched routes.

NAVIGATION

React Router provides tools to navigate between pages.

Link

- Renders an anchor tag (<a>), but prevents the page from reloading.
- Use to to specify the target route.

○ ○ ○

```
<Link to="/">Home</Link>
<Link to="/about">About</Link>
<Link to="/users/1">User 1</Link>
```

useNavigate

- A hook for programmatic navigation.
- Useful when you want to redirect the user after an action.

○ ○ ○

```
import { useNavigate } from 'react-router-dom';

function GoHome() {
  const navigate = useNavigate();

  const handleClick = () => {
    navigate('/');
  };

  return <button onClick={handleClick}>Go Home</button>;
}
```

DYNAMIC ROUTES

You can define routes with dynamic segments using a colon (:):

○○○

```
<Route path="/users/:id" element={<UserDetails />} />
```

Accessing Route Parameters

- Use the useParams hook to get the dynamic values.

○○○

```
import { useParams } from 'react-router-dom';

function UserDetails() {
  const { id } = useParams();
  return <h1>User ID: {id}</h1>;
}
```


NESTED ROUTES

Nested routes allow you to build a hierarchy of pages.

Setting Up Nested Routes

○ ○ ○

```
<Routes>
  <Route path="/users" element={<Users />}>
    <Route path=":id" element={<UserDetails />} />
    <Route path="settings" element={<UserSettings />} />
  </Route>
</Routes>
```

Rendering Nested Components with Outlet

- Use the Outlet component in the parent to render child components dynamically.

○ ○ ○

```
function Users() {
  return (
    <div>
      <h1>Users Page</h1>
      <Outlet />
    </div>
  );
}
```

REDIRECTING

React Router allows you to redirect users using the `Navigate` component.

Redirect Example

○○○

```
<Route path="/old-route" element={<Navigate to="/new-route" replace />} />
```

- The `replace` prop prevents the old route from being added to the browser history.

PROTECTED ROUTES

You can create protected routes by checking authentication or permissions.

Example: Protected Route

○○○

```
function ProtectedRoute({ element, isAuthenticated }) {  
  return isAuthenticated ? element : <Navigate to="/login" />;  
}  
  
<Routes>  
  <Route  
    path="/dashboard"  
    element={<ProtectedRoute element={<Dashboard />} isAuthenticated={true} />}  
  />  
</Routes>
```


QUERY PARAMETERS

React Router allows you to manage query parameters with the `useSearchParams` hook.

Getting Query Parameters

○○○

```
import { useSearchParams } from 'react-router-dom';

function SearchPage() {
  const [searchParams] = useSearchParams();
  const query = searchParams.get('query');
  return <h1>Search Query: {query}</h1>;
}
```

Updating Query Parameters

○○○

```
const [searchParams, setSearchParams] = useSearchParams();

setSearchParams({ query: 'react' });
```